

COMP3711: Design and Analysis of Algorithms

WU Jinjun

October 3, 2026

Contents

1	Prerequisite: Asymptotic Notations	3
2	Divide and Conquer	4
2.1	Binary Search	4
2.1.1	Exercise 1: Rotated Sorted Array	4
2.1.2	Exercise 2: Finding the Last 0	5
2.2	Mergesort & Inversion Count	6
2.2.1	Warm-up: Tower of Hanoi	6
2.2.2	Merge: Combining Two Sorted Halves	6
2.2.3	Mergesort Algorithm	7
2.2.4	Counting Inversions	8
2.3	Maximum Subarray	9
2.4	Multiplication and Master Theorem	11
2.4.1	Integer Multiplication (Karatsuba)	11
2.4.2	Master Theorem for Recurrences	11
3	Randomized Algorithms	13
3.1	Why Randomization?	13
3.2	Linearity of Expectation	13
3.3	Example: Hiring Problem	13
4	Quicksort	14
4.1	Deterministic Quicksort (Lomuto Partition)	14
4.2	Quicksort Analysis	14
4.3	Hoare's Partition (Alternative)	15
5	Heapsort	16
5.1	Heap Data Structure	16
5.2	Max-Heapify (Bubble-Down)	16
5.3	Build Max-Heap	16
5.4	Heapsort Algorithm	17
5.5	Priority Queue Operations (Applications of Heaps)	17

6	Linear-Time Sorting	19
6.1	Counting Sort	19
6.2	Radix Sort	19
6.3	Bucket Sort	20

1 Prerequisite: Asymptotic Notations

O, Ω, Θ are used to describe the asymptotic behavior of functions.

Definitions of Asymptotic Notations

- (a) **Big-O (Upper Bound):** $f(n) = O(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$$

- (b) **Big-Omega (Lower Bound):** $f(n) = \Omega(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$$

- (c) **Big-Theta (Tight Bound):** $f(n) = \Theta(g(n))$ means that there exist positive constants c_1, c_2 and n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all $n \geq n_0$.

$$0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$$

- (d) **Not Comparable:** $f(n)$ and $g(n)$ are not comparable if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist.

Asymptotic Growth Hierarchy (Memorize This Order)

constant < logarithmic < polynomial < exponential

$$\begin{aligned} O(1) &\subset O(\log \log n) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \\ &\subset O(n^2) \subset O(n^k) \subset O(2^n) \subset O(n!) \subset O(n^n) \end{aligned}$$

Common Mistakes with Asymptotic Notation

- $O(g(n))$ describes an **upper bound**, not necessarily tight. $n = O(n^2)$ is technically true but not useful.
- $\Omega(g(n))$ describes a **lower bound**. Best-case $\neq$ lower bound!
- $\Theta(g(n))$ requires **both** upper and lower bounds to match.
- Always drop constant factors and lower-order terms: $3n^2 + 5n + 7 = \Theta(n^2)$, not $\Theta(3n^2)$.

2 Divide and Conquer

Solve a problem of size n by breaking it into one or more smaller problems of size less than n . Solve the smaller problems recursively and combine their solutions to solve the large problem.

Three Steps of D&C

1. **Divide:** Split the problem into subproblems.
2. **Conquer:** Solve subproblems recursively (or directly if small enough).
3. **Combine:** Merge subproblem solutions into the original solution.

2.1 Binary Search

Given a sorted array A and a target value x , find the index of x (or -1 if not found).

Algorithm 1 Binary Search

```
function BINARYSEARCH( $A, x$ )
   $low \leftarrow 0$ 
   $high \leftarrow \text{length}(A) - 1$ 
  while  $low \leq high$  do
     $mid \leftarrow \left\lfloor \frac{low + high}{2} \right\rfloor$ 
    if  $A[mid] = x$  then
      return  $mid$ 
    else if  $A[mid] < x$  then
       $low \leftarrow mid + 1$ 
    else
       $high \leftarrow mid - 1$ 
    end if
  end while
  return  $-1$ 
end function
```

Binary Search Analysis

Let $T(n)$ be the number of comparisons on an array of size n .

- Recurrence: $T(n) \leq T\left(\frac{n}{2}\right) + 2$, with $T(1) = T(0) = 1$.
- Expand: $T(n) \leq T\left(\frac{n}{2}\right) + 2 \leq T\left(\frac{n}{4}\right) + 4 \leq \dots \leq T(1) + 2 \log_2 n = 1 + 2 \log_2 n$.
- Therefore: $T(n) = O(\log n)$.

When to Use Binary Search

Binary search works whenever you can discard half the search space with one comparison. The array need not be literally sorted—it just needs a “monotonic” property that lets you decide which half contains the answer.

2.1.1 Exercise 1: Rotated Sorted Array

Let $A[1..n]$ be a sorted array of n distinct numbers that has been rotated $n - k$ steps for some unknown integer $k \in [1, n - 1]$. That is, $A[1..k]$ is sorted increasing, $A[k + 1..n]$ is sorted increasing, and $A[n] < A[1]$.

Example: $n = 16$, $k = 10$:

$A = [9, 13, 16, 18, 19, 23, 28, 31, 37, 42, 0, 1, 2, 5, 7, 8]$

(1) Find the rotation index k in $O(\log n)$. ($A[k]$ is the maximum element.)

Algorithm 2 Find Rotated Index

```
function FINDROTATEDINDEX( $A, l, r$ )
     $mid \leftarrow \left\lfloor \frac{l + r}{2} \right\rfloor$ 
    if  $A[mid] > A[mid + 1]$  then
        return  $mid$ 
    else if  $A[mid] \geq A[l]$  then
        return FINDROTATEDINDEX( $A, mid + 1, r$ )
    else
        return FINDROTATEDINDEX( $A, l, mid - 1$ )
    end if
end function
```

(2) Search for x in $O(\log n)$.

Algorithm 3 Search in Rotated Sorted Array

```
function SEARCHROTATED( $A, x, n$ )
     $k \leftarrow$  FINDROTATEDINDEX( $A, 1, n$ )
    if  $x \geq A[1]$  then
        return BINARYSEARCH( $A[1..k], x$ )
    else
        return BINARYSEARCH( $A[k + 1..n], x$ )
    end if
end function
```

2.1.2 Exercise 2: Finding the Last 0

Array $A[1..n]$ contains 0s followed by 1s.

(1) Standard $O(\log n)$ solution.

Algorithm 4 Find Last 0

```
function FINDLAST0( $A, l, r$ )
     $mid \leftarrow \left\lfloor \frac{l + r}{2} \right\rfloor$ 
    if  $A[mid] = 0$  and  $A[mid + 1] = 1$  then
        return  $mid$ 
    else if  $A[mid] = 0$  then
        return FINDLAST0( $A, mid + 1, r$ )
    else
        return FINDLAST0( $A, l, mid - 1$ )
    end if
end function
```

(2) $O(\log k)$ solution when $k \ll n$. Exponential search (gallop): double the index until we overshoot, then binary search.

Algorithm 5 Find Last 0 with $O(\log k)$

```
 $i \leftarrow 1$ 
while  $i < n \wedge A[i] = 0$  do
     $i \leftarrow \min\{2i, n\}$ 
end while
if  $A[i] = 0$  then
    return  $i$ 
else if  $i = 1$  then
    return  $-1$ 
else
    return FINDLAST0( $A, \lfloor i/2 \rfloor, i$ )
end if
```

▷ all zeros: last 0 is n
▷ no zero

2.2 Mergesort & Inversion Count

2.2.1 Warm-up: Tower of Hanoi

Three pegs, n disks of decreasing size. Move all disks from source to target peg.

Hanoi Recurrence

Let $T(n)$ = minimum moves for n disks.

- Basis: $T(1) = 1$.
- Recursive step: move $n - 1$ disks source→aux, move largest source→target, move $n - 1$ aux→target.

$$T(n) = 2T(n - 1) + 1$$

- Solution: $T(n) = 2^n - 1$. (Expand: $T(n) = 2^k T(n - k) + (2^k - 1)$, set $k = n - 1$.)

Algorithm 6 Hanoi Tower

```
function HANOI( $n, A, B, C$ )
    if  $n = 1$  then
        Move disk from  $A$  to  $C$ 
    else
        HANOI( $n - 1, A, C, B$ )
        Move disk from  $A$  to  $C$ 
        HANOI( $n - 1, B, A, C$ )
    end if
end function
```

2.2.2 Merge: Combining Two Sorted Halves

Given two sorted subarrays $A[p..q]$ and $A[q + 1..r]$, merge them into $A[p..r]$ sorted.

Algorithm 7 Merge

```
function MERGE( $A, p, q, r$ )
   $n_1 \leftarrow q - p + 1$ ;    $n_2 \leftarrow r - q$ 
  Create arrays  $L[1..n_1 + 1]$ ,  $R[1..n_2 + 1]$ 
  for  $i = 1$  to  $n_1$  do
     $L[i] \leftarrow A[p + i - 1]$ 
  end for
  for  $j = 1$  to  $n_2$  do
     $R[j] \leftarrow A[q + j]$ 
  end for
   $L[n_1 + 1] \leftarrow \infty$ ;    $R[n_2 + 1] \leftarrow \infty$  ▷ sentinels
   $i \leftarrow 1$ ;    $j \leftarrow 1$ 
  for  $k = p$  to  $r$  do
    if  $L[i] \leq R[j]$  then
       $A[k] \leftarrow L[i]$ ;    $i \leftarrow i + 1$ 
    else
       $A[k] \leftarrow R[j]$ ;    $j \leftarrow j + 1$ 
    end if
  end for
end function
```

Merge Time Complexity

Merge runs in $\Theta(n)$ time on $n = r - p + 1$ elements: two copy loops ($n_1 + n_2 = n$) plus one comparison loop (n iterations). Sentinels avoid boundary checks — clean but not cache-friendly in practice.

2.2.3 Mergesort Algorithm

Algorithm 8 Mergesort

```
function MERGESORT( $A, p, r$ )
  if  $p < r$  then
     $q \leftarrow \left\lfloor \frac{p + r}{2} \right\rfloor$  ▷ Divide
    MERGESORT( $A, p, q$ ) ▷ Conquer left
    MERGESORT( $A, q + 1, r$ ) ▷ Conquer right
    MERGE( $A, p, q, r$ ) ▷ Combine
  end if
end function
```

Mergesort Analysis

Recurrence for n elements (power of 2 for simplicity):

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Solution: $T(n) = \Theta(n \log n)$. (Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$, case 2.)

Mergesort Properties

- **Stable:** yes (preserves order of equal keys if $L[i] \leq R[j]$ uses $\leq$, not $<$).
- **In-place:** no (needs auxiliary arrays L and R ; $\Theta(n)$ extra space).

- **Best / Worst / Average:** all $\Theta(n \log n)$ (unlike Quicksort).

2.2.4 Counting Inversions

An **inversion** is a pair (i, j) with $i < j$ and $A[i] > A[j]$. Count the total number of inversions.

Brute Force is Too Slow

Checking all $\binom{n}{2}$ pairs takes $\Theta(n^2)$ time. With D&C we can do $\Theta(n \log n)$ by **piggy-backing on Mergesort**.

Algorithm 9 Count Inversions via Mergesort

```

function COUNTINV( $A, p, r$ )
  if  $p \geq r$  then
    return 0
  end if
   $q \leftarrow \lfloor (p + r) / 2 \rfloor$ 
   $left \leftarrow \text{COUNTINV}(A, p, q)$ 
   $right \leftarrow \text{COUNTINV}(A, q + 1, r)$ 
   $cross \leftarrow \text{MERGECOUNT}(A, p, q, r)$ 
  return  $left + right + cross$ 
end function

function MERGECOUNT( $A, p, q, r$ )
  ... same as Merge, but whenever  $R[j]$  is copied before  $L[i]$  is exhausted:
  add  $(n_1 - i + 1)$  to  $cross$  (all remaining  $L[i..n_1]$  are  $> R[j]$ )
  return  $cross$ 
end function

```

Why Cross-Inversions Work

When copying $R[j]$ before $L[i]$ is empty, **every remaining element in L forms an inversion with $R[j]$** because L indices $< R$ indices (by array split) and $L[i..n_1] > R[j]$ (since we picked $R[j]$ first). This adds exactly $n_1 - i + 1$ inversions in $O(1)$.

2.3 Maximum Subarray

Given an array $A[1..n]$, find indices i, j maximizing $\sum_{t=i}^j A[t]$. (Useful for best-buy/best-sell stock problem.)

D&C Strategy

For a subarray $A[low..high]$ with midpoint mid , the maximum subarray is **exactly one** of:

1. Entirely in $A[low..mid]$ (recurse left).
2. Entirely in $A[mid + 1..high]$ (recurse right).
3. **Crossing the midpoint:** one end in left half, one end in right half (compute directly).

Return the maximum of the three.

Algorithm 10 Maximum Subarray (D&C)

```

function MAXSUBARRAY( $A, low, high$ )
    if  $low = high$  then
        return ( $low, high, A[low]$ )
    end if
     $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
    ( $left\_lo, left\_hi, left\_sum$ )  $\leftarrow$  MAXSUBARRAY( $A, low, mid$ )
    ( $right\_lo, right\_hi, right\_sum$ )  $\leftarrow$  MAXSUBARRAY( $A, mid + 1, high$ )
    ( $cross\_lo, cross\_hi, cross\_sum$ )  $\leftarrow$  MAXCROSSING( $A, low, mid, high$ )
    if  $left\_sum \geq right\_sum$  and  $left\_sum \geq cross\_sum$  then
        return ( $left\_lo, left\_hi, left\_sum$ )
    else if  $right\_sum \geq left\_sum$  and  $right\_sum \geq cross\_sum$  then
        return ( $right\_lo, right\_hi, right\_sum$ )
    else
        return ( $cross\_lo, cross\_hi, cross\_sum$ )
    end if
end function

function MAXCROSSING( $A, low, mid, high$ )
     $left\_sum \leftarrow -\infty$ ;  $sum \leftarrow 0$ 
    for  $i = mid$  downto  $low$  do
         $sum \leftarrow sum + A[i]$ 
        if  $sum > left\_sum$  then
             $left\_sum \leftarrow sum$ ;  $max\_left \leftarrow i$ 
        end if
    end for
     $right\_sum \leftarrow -\infty$ ;  $sum \leftarrow 0$ 
    for  $j = mid + 1$  to  $high$  do
         $sum \leftarrow sum + A[j]$ 
        if  $sum > right\_sum$  then
             $right\_sum \leftarrow sum$ ;  $max\_right \leftarrow j$ 
        end if
    end for
    return ( $max\_left, max\_right, left\_sum + right\_sum$ )
end function

```

Max Subarray Analysis

Recurrence:

$$T(n) = \begin{cases} \Theta(1) & n = 1, \\ 2T(n/2) + \Theta(n) & n > 1. \end{cases}$$

Same as Mergesort: $T(n) = \Theta(n \log n)$.

Kadane's Algorithm (iterative DP, not D&C): $O(n)$ time, $O(1)$ space.

$$\begin{aligned}\text{ending_here}_i &= \max(A[i], \text{ending_here}_{i-1} + A[i]), \\ \text{best}_i &= \max(\text{best}_{i-1}, \text{ending_here}_i).\end{aligned}$$

2.4 Multiplication and Master Theorem

2.4.1 Integer Multiplication (Karatsuba)

Given two n -bit integers x and y , compute $x \cdot y$.

Grade-School Multiplication

The standard pencil-and-paper algorithm is $\Theta(n^2)$: multiply each bit of y by x (shift & add), giving n rows of $O(n)$ bits each.

Karatsuba Split

Write $x = a \cdot 2^{n/2} + b$, $y = c \cdot 2^{n/2} + d$ (split into high/low halves). Then:

$$x \cdot y = ac \cdot 2^n + [(a+b)(c+d) - ac - bd] \cdot 2^{n/2} + bd.$$

Instead of 4 half-size multiplications (ac, ad, bc, bd), we use **3**: ac , bd , and $(a+b)(c+d)$, subtracting to recover $ad + bc$.

Algorithm 11 Karatsuba Multiplication

```

function MULTIPLY( $x, y, n$ )
  if  $n = 1$  then
    return  $x \cdot y$ 
  end if
   $a \leftarrow$  high  $n/2$  bits of  $x$ ;    $b \leftarrow$  low  $n/2$  bits of  $x$ 
   $c \leftarrow$  high  $n/2$  bits of  $y$ ;    $d \leftarrow$  low  $n/2$  bits of  $y$ 
   $ac \leftarrow$  MULTIPLY( $a, c, n/2$ )
   $bd \leftarrow$  MULTIPLY( $b, d, n/2$ )
   $ab\_cd \leftarrow$  MULTIPLY( $a + b, c + d, n/2$ )
  return  $ac \ll n + (ab\_cd - ac - bd) \ll (n/2) + bd$ 
end function

```

Karatsuba Recurrence

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n).$$

By Master Theorem case 1 ($\log_b a = \log_2 3 \approx 1.585 > 1$):

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}),$$

asymptotically better than $\Theta(n^2)$.

2.4.2 Master Theorem for Recurrences

For recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, b > 1.$$

Compare $f(n)$ with $n^{\log_b a}$ (the “leaf work”).

Master Theorem — Three Cases

1. **Case 1 (Leaves dominate):** If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$,

$$T(n) = \Theta(n^{\log_b a}).$$

2. **Case 2 (Balanced):** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$,

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$

3. **Case 3 (Root dominates):** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and the **regularity condition** $a f(n/b) \leq c f(n)$ holds for some $c < 1$ and large n ,

$$T(n) = \Theta(f(n)).$$

Common D&C Recurrences (Memorize!)

Algorithm	Recurrence	Solution
Binary search	$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$ (Case 2, $k = 0$)
Mergesort	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$ (Case 2, $k = 0$)
Karatsuba	$T(n) = 3T(n/2) + \Theta(n)$	$\Theta(n^{\log_2 3})$ (Case 1)
Strassen matrix	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7})$ (Case 1)
Bad D&C (naive 4-mult)	$T(n) = 4T(n/2) + \Theta(n)$	$\Theta(n^2)$ (Case 1)

3 Randomized Algorithms

3.1 Why Randomization?

A randomized algorithm flips coins during execution. Worst-case analysis becomes **expected**-case analysis, often beating deterministic worst-case bounds.

Two Types of Randomized Algorithms

- **Las Vegas:** always correct; running time is a random variable (e.g. randomized Quicksort). We bound the expected runtime.
- **Monte Carlo:** always fast; has a small probability of being wrong (e.g. randomized min-cut / Karger). Repeat to reduce error.

3.2 Linearity of Expectation

For any random variables $X_1, X_2, \dots, X_n$ (they need **not** be independent):

$$\mathbb{E} \left[\sum_{i=1}^n X_i \right] = \sum_{i=1}^n \mathbb{E}[X_i].$$

This is the single most useful tool for analyzing randomized algorithms.

Indicator Variables

To count “how many events happened”, set:

$$X_i = \mathbf{1}\{\text{event } i \text{ occurs}\} = \begin{cases} 1 & \text{if event } i \text{ happens,} \\ 0 & \text{otherwise.} \end{cases}$$

Then $\mathbb{E}[X_i] = \Pr(\text{event } i)$, and $X = \sum X_i$ gives the total count with $\mathbb{E}[X] = \sum \Pr(\text{event } i)$.

3.3 Example: Hiring Problem

Interview candidates in random order; hire whenever candidate is better than all previous. How many hires (expected)?

Let $X_i = 1$ if candidate i is hired (i.e. best among first i candidates). In random order this occurs with probability $1/i$.

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X_i] = \sum_{i=1}^n \frac{1}{i} = H_n = \Theta(\log n).$$

So the expected number of hires is the n -th harmonic number, $\ln n + O(1)$.

4 Quicksort

4.1 Deterministic Quicksort (Lomuto Partition)

Pick $A[r]$ as pivot; partition into $<$ pivot (left), $=$ pivot (middle), $>$ pivot (right).

Algorithm 12 Quicksort

```
function QUICKSORT( $A, p, r$ )
  if  $p < r$  then
     $q \leftarrow \text{PARTITION}(A, p, r)$ 
    QUICKSORT( $A, p, q - 1$ )
    QUICKSORT( $A, q + 1, r$ )
  end if
end function

function PARTITION( $A, p, r$ )
   $\text{pivot} \leftarrow A[r]$ 
   $i \leftarrow p - 1$ 
  for  $j = p$  to  $r - 1$  do
    if  $A[j] \leq \text{pivot}$  then
       $i \leftarrow i + 1$ 
      swap  $A[i] \leftrightarrow A[j]$ 
    end if
  end for
  swap  $A[i + 1] \leftrightarrow A[r]$ 
  return  $i + 1$ 
end function
```

Partition Invariants

At the end of each j -iteration in Partition:

- $A[p \dots i] \leq \text{pivot}$,
- $A[i + 1 \dots j] > \text{pivot}$,
- $A[j + 1 \dots r - 1]$ are unprocessed,
- $A[r] = \text{pivot}$.

Maintain with swaps; runtime is $\Theta(r - p + 1)$ on subarray size n .

4.2 Quicksort Analysis

Deterministic Quicksort Worst-Case

If the input is **already sorted** and pivot is always the **last** (or first) element:

$$T(n) = T(n - 1) + \Theta(n) \implies T(n) = \Theta(n^2).$$

Same as insertion sort! This is why we randomize.

Randomized Quicksort Expected Time

Pick pivot **uniformly at random** from $A[p..r]$ (swap it to position r before Partition). Let X = total number of **comparisons** made. Use indicator variables $X_{ij} = 1$ if elements of ranks i and j ($i < j$) are ever compared.

Key insight: ranks i and j are compared iff **the first pivot chosen from within** $\{i, i + 1, \dots, j\}$ **is either i or j** . That happens with probability $\frac{2}{j - i + 1}$.

$$\mathbb{E}[X] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} = \sum_{k=2}^n \frac{2}{k} \cdot (n - k + 1) = 2(n + 1)H_n - 4n.$$

So $\mathbb{E}[X] = \Theta(n \log n)$, and therefore expected runtime is $\Theta(n \log n)$.

Quicksort Properties

- **Stable:** no (Lomuto and Hoare partitions swap non-adjacent elements).
- **In-place:** yes (only $O(\log n)$ recursion stack; no auxiliary array).
- **Practical speed:** usually faster than Mergesort or Heapsort despite the same $\Theta(n \log n)$ bound, thanks to better cache locality and low constants.

4.3 Hoare's Partition (Alternative)

Two pointers moving toward each other from opposite ends; swap inversions. Does slightly fewer swaps than Lomuto. Pivot is not guaranteed to end at index q ; recur on $A[p..q]$ and $A[q + 1..r]$, **skipping** q .

5 Heapsort

5.1 Heap Data Structure

A (binary) **max-heap** is a complete binary tree satisfying:

$$A[\text{Parent}(i)] \geq A[i] \quad \forall i.$$

(Min-heap: $\leq$ instead.)

Array Representation of a Heap

Given 1-indexed array $A[1..n]$:

$$\text{Parent}(i) = \lfloor i/2 \rfloor,$$

$$\text{Left}(i) = 2i,$$

$$\text{Right}(i) = 2i + 1.$$

A complete binary tree on n nodes has height $\lfloor \lg n \rfloor$.

5.2 Max-Heapify (Bubble-Down)

Assume children of node i are heaps, but $A[i]$ may be too small. Fix by swapping $A[i]$ with the **larger** child, recursing down.

Algorithm 13 Max-Heapify

```
function MAXHEAPIFY( $A, i, \text{heap\_size}$ )
     $L \leftarrow 2i; \quad R \leftarrow 2i + 1$ 
     $\text{largest} \leftarrow i$ 
    if  $L \leq \text{heap\_size}$  and  $A[L] > A[\text{largest}]$  then
         $\text{largest} \leftarrow L$ 
    end if
    if  $R \leq \text{heap\_size}$  and  $A[R] > A[\text{largest}]$  then
         $\text{largest} \leftarrow R$ 
    end if
    if  $\text{largest} \neq i$  then
        swap  $A[i] \leftrightarrow A[\text{largest}]$ 
        MAXHEAPIFY( $A, \text{largest}, \text{heap\_size}$ )
    end if
end function
```

Time: $O(\log n)$ per call (proportional to tree height).

5.3 Build Max-Heap

Naive approach: run Max-Heapify top-down from the **last non-leaf** ($\lfloor n/2 \rfloor$) down to root 1. Leaves are trivially heaps of size 1.

Algorithm 14 Build Max-Heap

```
function BUILDMAXHEAP( $A, n$ )
    for  $i = \lfloor n/2 \rfloor$  downto 1 do
        MAXHEAPIFY( $A, i, n$ )
    end for
end function
```

Build-Max-Heap Cost is $O(n)$, Not $O(n \log n)$

Height- h nodes are $\leq n/2^{h+1}$; each MaxHeapify on a height- h node costs $O(h)$:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(2n) = O(n).$$

(Building from the leaves up is much cheaper than building from the root down.)

5.4 Heapsort Algorithm

(1) Build max-heap on $A[1..n]$. (2) Repeatedly: swap max (root $A[1]$) with last element, shrink heap, MaxHeapify root.

Algorithm 15 Heapsort

```
function HEAPSORT( $A, n$ )
    BUILDMAXHEAP( $A, n$ )
    for  $i = n$  downto 2 do
        swap  $A[1] \leftrightarrow A[i]$ 
         $heap\_size \leftarrow i - 1$ 
        MAXHEAPIFY( $A, 1, heap\_size$ )
    end for
end function
```

Heapsort Analysis

- Build: $O(n)$.
- $n - 1$ swaps each followed by MaxHeapify: $(n - 1) O(\log n) = O(n \log n)$.
- **Total:** $O(n \log n)$ (best = worst = average, unlike Quicksort).
- **Stable?** No.
- **In-place?** Yes (only $O(1)$ extra storage, no recursion stack if MaxHeapify is written iteratively).

5.5 Priority Queue Operations (Applications of Heaps)

Max-priority queue supports: Maximum, ExtractMax, IncreaseKey, Insert.

Algorithm 16 Priority Queue Operations

```
function EXTRACTMAX( $A, heap\_size$ )
     $max \leftarrow A[1]$ 
     $A[1] \leftarrow A[heap\_size]; \quad heap\_size \leftarrow heap\_size - 1$ 
    MAXHEAPIFY( $A, 1, heap\_size$ )
    return  $max$ 
end function

function INCREASEKEY( $A, i, key$ )
    if  $key < A[i]$  then error
    end if
     $A[i] \leftarrow key$ 
    while  $i > 1$  and  $A[\text{Parent}(i)] < A[i]$  do                                 $\triangleright$  bubble up
        swap  $A[i] \leftrightarrow A[\text{Parent}(i)]$ 
         $i \leftarrow \text{Parent}(i)$ 
    end while
end function

function INSERT( $A, key, heap\_size$ )
     $heap\_size \leftarrow heap\_size + 1$ 
     $A[heap\_size] \leftarrow -\infty$ 
    INCREASEKEY( $A, heap\_size, key$ )
end function
```

Each operation runs in $O(\log n)$ time.

6 Linear-Time Sorting

All comparison-based sorts have *worst-case* $\Omega(n \log n)$ lower bound (decision-tree height). The following beat the bound by **not comparing keys directly**.

When Can We Beat $\Omega(n \log n)$?

We need extra information about the keys: their values lie in a **small integer range**, or they have structure that allows *digit-wise* processing.

6.1 Counting Sort

Assume each of n input keys is an integer in $\{0, 1, \dots, k\}$.

Algorithm 17 Counting Sort

```
function COUNTINGSORT( $A, B, n, k$ )
  for  $i = 0$  to  $k$  do
     $C[i] \leftarrow 0$ 
  end for
  for  $j = 1$  to  $n$  do
     $C[A[j]] \leftarrow C[A[j]] + 1$                                 ▷ count frequencies
  end for
  for  $i = 1$  to  $k$  do
     $C[i] \leftarrow C[i] + C[i - 1]$                                 ▷ prefix sums = “how many  $\leq i$ ”
  end for
  for  $j = n$  downto  $1$  do
     $B[C[A[j]]] \leftarrow A[j]$ 
     $C[A[j]] \leftarrow C[A[j]] - 1$ 
  end for
end function
```

Counting Sort Properties

- Runtime: $\Theta(n + k)$. If $k = O(n)$ then $\Theta(n)$ overall.
- **Stable**: yes (fill B *right-to-left* so equal keys preserve input order). This is critical for being used as a subroutine in Radix Sort.
- **In-place**: no (uses arrays C of size $k + 1$ and output B of size n).

6.2 Radix Sort

Sort d -digit keys by applying a **stable** sort digit-by-digit, least significant digit first (LSD). Each digit sort is Counting Sort on that digit.

Algorithm 18 Radix Sort

```
function RADIXSORT( $A, d, n$ )
  for  $i = 1$  to  $d$  do
    Use a stable sort to sort  $A$  on digit  $i$  (the  $i$ -th least significant)
  end for
end function
```

Radix Sort Correctness & Complexity

- Why LSD works: after pass i , numbers are sorted correctly w.r.t. their low i digits;

next pass (higher digit) is stable so ties keep the lower-order ordering.

- Each digit in $\{0, \dots, k-1\}$ sorted by Counting Sort in $\Theta(n+k)$.
- Total: $\Theta(d(n+k))$. If d is constant and $k = O(n)$, this is $\Theta(n)$.
- **Stable:** yes (because inner sort is stable).

Radix Sort Gotchas

- Must process digits **from LSD to MSD** (not the other way around).
- Inner sort **must be stable**; otherwise equal-digit elements lose their previously-computed order.

6.3 Bucket Sort

Assume input is drawn **uniformly at random** from $[0, 1)$. Distribute into n equal-sized buckets, sort each bucket (e.g. insertion sort), concatenate.

Algorithm 19 Bucket Sort

```

function BUCKETSORT( $A, n$ )
    Create  $B[0..n-1]$  (each an empty list)
    for  $i = 1$  to  $n$  do
        insert  $A[i]$  into  $B[\lfloor n \cdot A[i] \rfloor]$ 
    end for
    for  $i = 0$  to  $n-1$  do
        sort list  $B[i]$  (e.g. insertion sort)
    end for
    Concatenate  $B[0], B[1], \dots, B[n-1]$  in order
end function

```

Bucket Sort Expected Time

With uniform input, each bucket has $O(1)$ elements in expectation; sorting each by insertion sort is $O(1)$ per bucket.

$$\mathbb{E}[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(1) = \Theta(n).$$

Worst case (all elements fall in one bucket): $\Theta(n^2)$ if inner sort is insertion sort.

Sorting Algorithm Cheat Sheet

Algorithm	Worst	Avg	Best	Stable	In-place	Notes
Insertion	n^2	n^2	n	yes	yes	good if nearly sorted
Mergesort	$n \log n$	$n \log n$	$n \log n$	yes	no	needs $\Theta(n)$ extra
Heapsort	$n \log n$	$n \log n$	$n \log n$	no	yes	worst-case guarantee
Quicksort	n^2	$n \log n$	$n \log n$	no	yes	fastest in practice
Counting	$n+k$	$n+k$	$n+k$	yes	no	keys in $[0, k]$
Radix	$d(n+k)$	$d(n+k)$	$d(n+k)$	yes	no	d -digit keys
Bucket	n^2	$\mathbf{E}[n]$	n	yes	no	uniform on $[0, 1)$